

Symbolica Integrate

Symbolic integration with genuine Rubi steps

A focused companion to Symbolica: pass in portable Symbolica Atom payloads, receive an antiderivative or the complete nested rule trace.

The official integration plugin, powered by Symbolica 3.0. Free for any use within Typst. No license or license key needed.

Repository · MIT license

1 Use Rubi from Symbolica

Symbolica Integrate is a separate package and WebAssembly engine. It deliberately does not parse expressions, declare symbols, or print mathematics. Symbolica and compatible extensions create the versioned Atom payloads it consumes and can continue working with the result.

It is free to use for any use within Typst, including academic and commercial work. No Symbolica license or license key is needed.

Import the Rubi package explicitly alongside Symbolica:

```
#import "@local/symbolica:0.1.0" as sym
#import "@local/symbolica-integrate:0.1.0": integrate, integrate-with-steps

#let x = sym.math($x$)
#let f = sym.math($x / (x + 1)$)
#let primitive = integrate(f, x)

$ integral #sym.to-tyrst(f) dif x = #sym.to-tyrst(primitive) + C $
```

The first integration call prepares Rubi's rules through Typst's cached plugin transition. Later calls and ordinary edits reuse the prepared module. Additional instances receive its initialized memory snapshot. Restarting the compiler or losing its cache requires preparation again.

1.1 A worked rule trace

$$\int x(1+x)^{-1} dx = x - \log(1+x) + C$$

Rule 49 : Algebraic expansion

$$\int x(1+x)^{-1} dx = \int 1 - (1+x)^{-1} dx$$

Rule 2009 : Integrate the terms of the sum separately.

$$\int 1 - (1+x)^{-1} dx = \int 1 dx + \int (-1-x)^{-1} dx$$

Rule 24 : Apply the direct antiderivative formula.

$$\int 1 dx = x$$

Rule 16 : Apply the direct antiderivative formula.

$$\int -1(1+x)^{-1} dx = -1 \log(1+x)$$

complete is true when Rubi eliminated every integral. A partial result is still returned when it is false. No integration constant is added. Symbol namespaces, attributes, and portable attachments are retained in the result and in each step payload.

2 Public API

The package exports exactly two public functions.

Function	Result
<code>integrate(expression, variable)</code>	The best-effort antiderivative as portable Atom payload bytes.
<code>integrate-with-steps(expression, variable)</code>	A dictionary containing result, complete, and nested steps.

Both arguments must be portable Symbolica Atom payload bytes, and `variable` must encode one symbol. Invalid input produces an error instead of being reparsed from its printed form. Thus `math($x$)` is a valid variable, while the annotated Typst content returned directly by `symbol("x")` must first be passed through Symbolica's `atom` function.

2.1 Step fields

Each step contains:

- rule: the Rubi rule number, or none for an auxiliary transformation;
- depth: nesting depth of the rewritten integral;
- description, references, and source: Rubi's explanatory metadata; and
- input and output: portable Atom payload bytes for the immediate rewrite.

The array is ordered from the outer rewrite into nested integrals. Use the depth field to indent it, group it, or build a custom explanation layout.

3 Installation from this repository

Install `symbolica` and `symbolica-integrate` as independent local packages. On Linux, expose the repository root as `Symbolica` and its `symbolica-integrate` directory as the integration package:

```
mkdir -p ~/.local/share/typst/packages/local/symbolica
mkdir -p ~/.local/share/typst/packages/local/symbolica-integrate
ln -s /path/to/symbolica-typst-plugin \
    ~/.local/share/typst/packages/local/symbolica/0.1.0
ln -s /path/to/symbolica-typst-plugin/symbolica-integrate \
    ~/.local/share/typst/packages/local/symbolica-integrate/0.1.0
```

The integration directory contains its own compressed engine and inflater. The rule tables are constructed at runtime instead of being stored in the download. Documents that use only `Symbolica` do not construct this engine.

`Symbolica Integrate` uses `Symbolica`'s integration interface and the Rubi ruleset. Thanks to the `Symbolica` and Rubi contributors for making the engine possible.